arXiv:submit/7819847 [cs.LG] 13 Jul 2026

# EXPLICIT ADJOINT EXPOSURE AND DECOUPLED AUTO-DIFFERENTIATION

**Vladimer Khasia**
Independent Researcher
vladimer.khasia.1@gmail.com

**ABSTRACT**

Standard reverse-mode automatic differentiation (AD) relies on a monolithic, global computational graph that records the entire network execution. This design requires maintaining a continuous activation tape, resulting in a multiplicative $\mathcal{O}(ND_{inner}L)$ spatial complexity while keeping intermediate gradients (adjoint states) inaccessible within the compiler backend. We propose **Explicit Adjoint Exposure (EAE)**, a training framework grounded in the discrete-time Adjoint State Method that **bypasses the global monolithic AD tape completely**. By reformulating layer-wise forward and backward sweeps as local Hamiltonian systems, EAE decouples the global computational graph, replacing it with a sequence of isolated, block-local AD sweeps that materialize the adjoint states (costates) in user manipulable way at block boundaries. This explicit exposure unlocks structural optimization opportunities inaccessible to standard backpropagation, including localized adjoint quantization and sparsification, dynamic intermediate gradient regularization, and synthetic costate modeling. Furthermore, by pairing this decoupled structure with aggressive local garbage collection (EAE-GC), we convert the backward spatial complexity to an additive $\mathcal{O}(NDL) + \mathcal{O}(ND_{inner})$ footprint, establishing a strict $\mathcal{O}(1)$ active tape memory scaling with respect to network depth. Empirical evaluations on an autoregressive transformer verify that EAE-GC preserves analytical gradient equivalence and equivalent convergence dynamics within floating-point precision limits while strictly reducing VRAM requirements.

The code is available at https://github.com/VladimerKhasia/eae

## 1 Introduction

The standard training of deep neural networks relies universally on the reverse-mode auto-differentiation algorithm, widely known as backpropagation. Originally established through independent derivations across distinct computational disciplines [1, 2, 3], and subsequently first used for specialized neural network architectures that exploit image structure through local connectivity and weight sharing [4], backpropagation computes exact loss sensitivities by recursively traversing a globally maintained computational graph.

Despite its computational efficiency, modern deep learning frameworks implement backpropagation as an opaque, monolithic engine. This design dictates that the forward activation tape must remain continuously in memory, yielding a multiplicative $\mathcal{O}(ND_{inner}L)$ spatial complexity. While standard gradient checkpointing mitigates this memory footprint via activation recomputation, it fundamentally preserves the black-box nature of the backward sweep. Consequently, the intermediate dual momenta—the adjoint states—remain strictly encapsulated within the auto-differentiation compiler, prohibiting localized gradient manipulation or inspection during the backward pass.

To resolve this opacity, we introduce **Explicit Adjoint Exposure (EAE)**, a framework grounded in the classical Discrete-Time Adjoint State Method derived from Pontryagin's Minimum Principle[5, 6]. **EAE bypasses the global monolithic AD tape entirely.** Rather than treating the backward pass as a single continuous graph, EAE reformulates the layer-wise forward and backward sweeps as canonical equations of motion of a discrete dynamical system. By isolating the computation into local layer Hamiltonians, EAE decouples the global computational graph, replacing it with a sequence of isolated, block-local AD sweeps that explicitly materialize the adjoint states (costates) at block boundaries directly in user accessible way.

Exposing these costates unlocks several core structural opportunities that standard backpropagation and traditional gradient checkpointing cannot naturally accommodate: (1) adjoint quantization and sparsification to significantly reduce memory and communication bandwidth; (2) dynamic intermediate regularization directly applied to local gradients without altering the global loss; (3) the integration of decoupled or synthetic adjoints to bypass sequential backward dependencies and so on.

Furthermore, we demonstrate that pairing this decoupled architecture with aggressive local garbage collection (EAE-GC) optimizes memory allocation dynamically. By dropping redundant intermediate tape allocations instantaneously at layer boundaries, EAE converts the spatial complexity of the active auto-differentiation engine from a multiplicative factor to an additive one. This establishes a strict $\mathcal{O}(1)$ scaling of the active tape memory relative to continuous graph depth, reducing the total backward memory footprint to an additive $\mathcal{O}(NDL) + \mathcal{O}(ND_{inner})$ bound. Under empirical evaluation, EAE preserves analytical gradient equivalence to standard backpropagation, yielding matching convergence dynamics within standard floating-point precision limits and safely bridging the gap between extreme memory efficiency and highly manipulable model training.

## 2 Methodology

In this section, we formalize the **Explicit Adjoint Exposure (EAE)** framework. Rather than treating the backward pass of a neural network as an opaque, monolithic engine, we ground our framework in the classical **Discrete-Time Adjoint State Method** (derived from Pontryagin's Minimum Principle). By reformulating the layer-wise forward and backward sweeps as canonical equations of motion of a discrete dynamical system, we decouple the global computational graph. This decoupling allows us to explicitly materialize the adjoint states (dual momenta) in user accessible way, unlocking a new class of intermediate optimizations while naturally maintaining the $\mathcal{O}(1)$ active tape memory scaling of gradient checkpointing.

### 2.1 Optimal Control and the Discrete-Time Hamiltonian

Let an $L$-layer neural network be modeled as a discrete-time dynamical system. The forward propagation of an input $\mathbf{x}_0 \in \mathcal{X}_0 \subseteq \mathbb{R}^{B\times S\times D}$ is governed by the state transition equation:

$$\mathbf{x}_l = h_l(\mathbf{x}_{l-1}, \mathbf{W}_l), \quad l \in \{1, \dots, L\} \tag{1}$$

where $\mathbf{x}_l \in \mathbb{R}^{B\times S\times D}$ represents the state tensor of layer $l$, and $\mathbf{W}_l \in \Theta_l$ denotes the parameter tensor of block $l$.

To optimize the system parameters $\{\mathbf{W}_l\}_{l=1}^{L}$ with respect to a terminal loss functional $\mathcal{L}$, we introduce the adjoint variables $\mathbf{p}_l \in \mathbb{R}^{B\times S\times D}$.

**Definition 2.1** (Adjoint State)**.** The adjoint state $\mathbf{p}_l$ associated with the primal state $\mathbf{x}_l$ is defined as the exact sensitivity of the global loss with respect to that state:

$$\mathbf{p}_l \triangleq \nabla_{\mathbf{x}_l}\mathcal{L} \tag{2}$$

We formulate the optimization of each discrete transition step by introducing the Discrete Layer Hamiltonian.

**Definition 2.2** (Discrete Layer Hamiltonian)**.** For any layer $l \in \{1, \dots, L\}$, given the incoming state $\mathbf{x}_{l-1}$ and the costate (adjoint state) $\mathbf{p}_l$, the Discrete Layer Hamiltonian $\mathcal{H}_l : \Theta_l \times \mathcal{X}_{l-1} \to \mathbb{R}$ is defined as:

$$\mathcal{H}_l(\mathbf{x}_{l-1}, \mathbf{W}_l; \mathbf{p}_l) = \langle h_l(\mathbf{x}_{l-1}, \mathbf{W}_l), \mathbf{p}_l \rangle \tag{3}$$

where $\langle \cdot, \cdot \rangle$ denotes the standard Euclidean inner product over the tensor space (equivalent to the sum of the element-wise Hadamard product).

**Proposition 2.1** (Exact Adjoint Equivalence)**.** *The gradients of the global loss functional $\mathcal{L}$ with respect to the intermediate state variables and the layer parameters are governed by the canonical equations of the Discrete Hamiltonian:*

$$\mathbf{p}_{l-1} = \nabla_{\mathbf{x}_{l-1}}\mathcal{H}_l \equiv \mathcal{J}_{h_l}^{\top}(\mathbf{x}_{l-1})\mathbf{p}_l \tag{4}$$

$$\nabla_{\mathbf{W}_l}\mathcal{L} = \nabla_{\mathbf{W}_l}\mathcal{H}_l \equiv \mathcal{J}_{h_l}^{\top}(\mathbf{W}_l)\mathbf{p}_l \tag{5}$$

*where $\mathcal{J}_{h_l}^{\top}(\mathbf{u})\mathbf{v}$ denotes the Vector-Jacobian Product (VJP) mapping of the transformation $h_l$ with respect to the tensor variable $\mathbf{u}$ evaluated along the vector $\mathbf{v}$.*

*Proof.* Let the total derivative of the loss $\mathcal{L}$ be expressed recursively. Applying the multivariable chain rule to $\mathcal{L}$ with respect to the tensor $\mathbf{x}_{l-1}$ yields the standard reverse-mode auto-differentiation identity:

$$\nabla_{\mathbf{x}_{l-1}}\mathcal{L} = \mathcal{J}_{h_l}^{\top}(\mathbf{x}_{l-1})\nabla_{\mathbf{x}_l}\mathcal{L} \tag{6}$$

Substituting $\mathbf{p}_l \triangleq \nabla_{\mathbf{x}_l} \mathcal{L}$ and evaluating the gradient of the Hamiltonian inner product $\mathcal{H}_l$ with respect to the input state $\mathbf{x}_{l-1}$ confirms that $\nabla_{\mathbf{x}_{l-1}} \mathcal{H}_l = \mathcal{J}_{h_l}^\top(\mathbf{x}_{l-1})\mathbf{p}_l$, satisfying (4). An identical contraction with respect to the parameter tensor $\mathbf{W}_l$ yields (5), completing the validation. □

### 2.2 Adjoint Boundary Condition

The terminal loss is defined over the unnormalized logit space $\mathbf{z} \in \mathbb{R}^{B \times S \times |\mathcal{V}|}$. To ensure exact alignment with standard transformer architectures, the primal output of the final block $\mathbf{x}_L \in \mathbb{R}^{B \times S \times D}$ must pass through a layer normalization before projection:

$$\bar{\mathbf{x}}_L = \text{RMSNorm}(\mathbf{x}_L; \boldsymbol{\gamma}) \tag{7}$$

$$\mathbf{z} = \bar{\mathbf{x}}_L \mathbf{W}_{head}^\top \tag{8}$$

where $\boldsymbol{\gamma} \in \mathbb{R}^D$ is the learnable normalization scale parameter and $\mathbf{W}_{head} \in \mathbb{R}^{|\mathcal{V}| \times D}$ is the classification head. For a Cross-Entropy loss $\mathcal{L} = \text{CE}(\mathbf{z}, \mathbf{y})$, the boundary sensitivity at the logit level is given by:

$$\mathbf{g} = \nabla_{\mathbf{z}} \mathcal{L} = \frac{1}{B \cdot S} \left( \text{Softmax}(\mathbf{z}) - \mathbf{y}_{\text{one-hot}} \right) \tag{9}$$

To map this vocabulary-space gradient back to the hidden space of the terminal block, we execute the sequential Vector-Jacobian Products through the head projection and the normalization layer:

$$\bar{\mathbf{p}}_L \triangleq \nabla_{\bar{\mathbf{x}}_L} \mathcal{L} = \mathbf{g} \mathbf{W}_{head} \tag{10}$$

$$\mathbf{p}_L \triangleq \nabla_{\mathbf{x}_L} \mathcal{L} = \mathcal{J}_{\text{RMSNorm}}^\top(\mathbf{x}_L) \bar{\mathbf{p}}_L \tag{11}$$

The corresponding parameter gradients for the normalization scale $\boldsymbol{\gamma}$ and head projection $\mathbf{W}_{head}$ are computed via:

$$\nabla_{\mathbf{W}_{head}} \mathcal{L} = \mathbf{g}^\top \bar{\mathbf{x}}_L \tag{12}$$

$$\nabla_{\boldsymbol{\gamma}} \mathcal{L} = \mathcal{J}_{\text{RMSNorm}}^\top(\boldsymbol{\gamma}) \bar{\mathbf{p}}_L \tag{13}$$

We encapsulate these coupled terminal projections into a single boundary operator $\mathcal{B}(\mathbf{x}_L, \mathbf{y}; \mathbf{W}_{head}, \boldsymbol{\gamma})$. This formulation resolves the dimensional mismatch inherent to naive vocabulary-space computations, ensuring mathematically exact boundary conditions for the costate backward propagation.

### 2.3 Algorithm Specification

Algorithm 1 details the foundational Explicit Adjoint Exposure (EAE) method, which severs the global autograd graph while retaining intermediate states. Algorithm 2 specifies the memory-optimized EAE (EAE-GC) utilizing aggressive local garbage collection.

**Algorithm 1** Explicit Adjoint Exposure (EAE)

**Require:** Data batch $(\mathbf{x}_{raw}, \mathbf{y})$, Blocks $h_1 \dots h_L$, Parameters $\mathbf{W}_1 \dots \mathbf{W}_L$, Embedding $\mathbf{W}_{emb}$, Head parameters $\mathbf{W}_{head}, \boldsymbol{\gamma}$

**Phase 1: Forward State Sweep (Detached)**
$\mathbf{x}_0 \leftarrow \text{Embed}(\mathbf{x}_{raw}, \mathbf{W}_{emb})$
Initialize $\mathcal{S}_{primal} \leftarrow [\mathbf{x}_0]$
**for** $l = 1$ to $L$ **do**
  $\mathbf{x}_l \leftarrow h_l(\mathbf{x}_{l-1}, \mathbf{W}_l)$ {Computed with no grad tape}
  $\mathcal{S}_{primal}$.append($\mathbf{x}_l$)
**end for**
**Phase 2: Adjoint Boundary Extraction**
$\mathbf{x}_L \leftarrow \mathcal{S}_{primal}[L]$.requires_grad_()
$\mathcal{L}_{bound} \leftarrow \mathcal{B}(\mathbf{x}_L, \mathbf{y}; \mathbf{W}_{head}, \boldsymbol{\gamma})$
$\mathbf{p}_L, \Delta\mathbf{W}_{head}, \Delta\boldsymbol{\gamma} \leftarrow \nabla_{\mathbf{x}_L, \mathbf{W}_{head}, \boldsymbol{\gamma}} \mathcal{L}_{bound}$
$\mathbf{W}_{head}$.grad $\leftarrow \mathbf{W}_{head}$.grad $+ \Delta\mathbf{W}_{head}$
$\boldsymbol{\gamma}$.grad $\leftarrow \boldsymbol{\gamma}$.grad $+ \Delta\boldsymbol{\gamma}$
**Phase 3: Sequential Adjoint Propagation**
$\mathbf{p}_{curr} \leftarrow \mathbf{p}_L$.detach()
**for** $l = L$ down to 1 **do**
  $\mathbf{x}_{l-1} \leftarrow \mathcal{S}_{primal}[l-1]$.requires_grad_()
  $\mathbf{y}_{local} \leftarrow h_l(\mathbf{x}_{l-1}, \mathbf{W}_l)$ {Reconstruct local graph}
  $\mathcal{H}_l \leftarrow \langle \mathbf{y}_{local}, \mathbf{p}_{curr} \rangle$ {Discrete Hamiltonian evaluation}
  $\mathbf{p}_{curr}, \Delta\mathbf{W}_l \leftarrow \nabla_{\mathbf{x}_{l-1}, \mathbf{W}_l} \mathcal{H}_l$
  $\mathbf{W}_l$.grad $\leftarrow \mathbf{W}_l$.grad $+ \Delta\mathbf{W}_l$
  $\mathbf{p}_{curr} \leftarrow \mathbf{p}_{curr}$.detach()
**end for**
**Phase 4: Embedding Adjoint Matching**
$\mathbf{x}_{emb} \leftarrow \text{Embed}(\mathbf{x}_{raw}, \mathbf{W}_{emb})$
$\mathcal{H}_{emb} \leftarrow \langle \mathbf{x}_{emb}, \mathbf{p}_{curr} \rangle$
$\Delta\mathbf{W}_{emb} \leftarrow \nabla_{\mathbf{W}_{emb}} \mathcal{H}_{emb}$
$\mathbf{W}_{emb}$.grad $\leftarrow \mathbf{W}_{emb}$.grad $+ \Delta\mathbf{W}_{emb}$

**Algorithm 2** Memory-Optimized EAE (EAE-GC)

**Require:** Same inputs as Algorithm 1.
Execute **Phase 1** and **Phase 2** identically to Algorithm 1.
**Phase 3: Sequential Adjoint Propagation with Aggressive GC**
$\mathbf{p}_{curr} \leftarrow \mathbf{p}_L$.detach()
**for** $l = L$ down to 1 **do**
  $\mathbf{x}_{l-1} \leftarrow \mathcal{S}_{primal}[l-1]$.requires_grad_()
  $\mathbf{y}_{local} \leftarrow h_l(\mathbf{x}_{l-1}, \mathbf{W}_l)$
  $\mathcal{H}_l \leftarrow \langle \mathbf{y}_{local}, \mathbf{p}_{curr} \rangle$
  $\mathbf{p}_{new}, \Delta\mathbf{W}_l \leftarrow \nabla_{\mathbf{x}_{l-1}, \mathbf{W}_l} \mathcal{H}_l$
  $\mathbf{W}_l$.grad $\leftarrow \mathbf{W}_l$.grad $+ \Delta\mathbf{W}_l$
  $\mathbf{p}_{curr} \leftarrow \mathbf{p}_{new}$.detach()
  **del** $\mathbf{y}_{local}$, $\mathcal{H}_l$, $\mathbf{p}_{new}$, $\mathbf{x}_{l-1}$ {Instantaneous VRAM deallocation}
**end for**
Execute **Phase 4** identically to Algorithm 1.

**Implementation Details: Weight Tying and Mixed Precision.** In architectures employing weight tying ($\mathbf{W}_{head} \equiv \mathbf{W}_{emb}$), the gradients evaluated for the head projection in Phase 2 and the initial embedding layer in Phase 4 are accumulated directly into a shared physical tensor. Furthermore, to ensure numerical stability and mathematically correct accumulation across micro batches, the empirical boundary operator $\mathcal{B}$ incorporates a scaling factor $\eta/K$, where $\eta$ is the Automatic Mixed Precision (AMP) loss scale (preventing dual momentum underflow in FP16) and $K$ is the number of gradient accumulation steps.

### 2.4 Complexity Analysis and the Value Proposition of Explicit Adjoint Exposure

While standard gradient checkpointing yields similar asymptotic spatial limits by recomputing activations, it treats the backward pass as an inaccessible black box. Our framework, by contrast, explicitly materializes the costates (adjoint variables) $\mathbf{p}_l$ at block boundaries.

**Lemma 2.2** (Space Complexity Bound)**.** *Standard BP exhibits a spatial complexity of $\mathcal{O}(ND_{inner}L)$, where $D_{inner}$ encapsulates intermediate activations (e.g., Attention matrices, SwiGLU expansions). Algorithm 2 restricts the active auto-differentiation tape spatial complexity to $\mathcal{O}(ND_{inner})$, achieving $\mathcal{O}(1)$ scaling with respect to depth $L$.*

*Proof.* In standard BP, the global chain rule dictates that the forward tape must remain continuously in memory. Every intermediate operation $\mathbf{z}_{l,i}$ within $h_l$ must be preserved until the backward pass traverses it, defining an activation memory footprint $\mathcal{M}_{BP} = \sum_{l=1}^{L} \mathcal{O}(ND_{inner}) = \mathcal{O}(ND_{inner}L)$.

In Algorithm 2 (EAE-GC), Phase 1 is executed under a detached context (`torch.no_grad()`), allocating strictly the boundary states $\mathcal{S}_{primal}$ which requires $\mathcal{O}(NDL)$ space. Crucially, the massive intermediate matrices proportional to $D_{inner}$ are discarded. During Phase 3, a local computational graph is instantiated to evaluate $\mathcal{H}_l$. The memory footprint of this local graph is $\mathcal{O}(ND_{inner})$. However, upon execution of the garbage collection step (**del** $\mathbf{y}_{local}$, $\mathcal{H}_l$), this graph is annihilated before the loop increments to $l-1$.

Therefore, the peak memory allocated to the dynamic auto-differentiation engine at any given timestep $t$ is strictly decoupled from the global depth $L$, resolving to $\mathcal{O}(ND_{inner})$. The total spatial complexity scales as:

$$\mathcal{M}_{EAE-GC} = \mathcal{O}(NDL) + \mathcal{O}(ND_{inner}) \tag{14}$$

Because $D_{inner} \gg D$ in modern architectures, our approach successfully replaces the multiplicative memory factor $\mathcal{O}(ND_{inner}L)$ with an additive one, establishing an $\mathcal{O}(1)$ tape scaling relative to continuous graph depth. □

**Lemma 2.3** (Time Complexity Equivalence)**.** *Both EAE variants share asymptotic time complexity $\mathcal{O}(T)$ with standard BP, incurring an exact theoretical scalar compute overhead of $1.33\times$.*

*Proof.* Let $C_f$ be the FLOPs required for one forward pass of layer $h_l$, and $C_b = 2C_f$ be the FLOPs for the corresponding VJPs in standard BP. The total BP compute is $\mathcal{O}(L \cdot (C_f + C_b)) = \mathcal{O}(3LC_f)$. EAE algorithms require an initial detached forward sweep (Phase 1, $\mathcal{O}(LC_f)$), followed by a local recomputation and backward mapping (Phase 3, $\mathcal{O}(L \cdot (C_f + C_b))$). Total compute is exactly $\mathcal{O}(L \cdot (2C_f + 2C_f)) = \mathcal{O}(4LC_f)$. Asymptotically, $\mathcal{O}(4LC_f) \equiv \mathcal{O}(3LC_f)$, confirming strict $\mathcal{O}(N \cdot D^2 \cdot L)$ linear time scalability identical to BP, differing only by a constant recomputation factor universally analogous to standard gradient checkpointing. □

#### 2.4.1 Advantages of EAE

By explicitly exposing the costate variable $\mathbf{p}_l$ at the block boundary in Python memory, EAE unlocks three core structural opportunities that standard backpropagation cannot naturally accommodate:

1. **Adjoint Quantization and Sparsification:** Because $\mathbf{p}_l$ is a decoupled tensor, researchers can apply quantization (e.g., FP8, INT4 compression) or top-$k$ sparsification to the adjoint states before propagating them to layer $l-1$, substantially saving communication and memory bandwidth.
2. **Intermediate Regularization:** Arbitrary loss penalties, clipping operations, or custom projections can be dynamically applied directly to the costates $\mathbf{p}_l$ without altering the global loss $\mathcal{L}$.
3. **Decoupled and Synthetic Adjoints:** EAE provides the mathematical foundation for predicting synthetic costates $\tilde{\mathbf{p}}_l \approx \mathbf{p}_l$ using lightweight auxiliary networks, potentially breaking the remaining sequential loop-carry dependency during training.

## 3 Experiments

The empirical evaluation of our framework is designed to validate two core theoretical claims: (1) the mathematical equivalence of the Explicit Adjoint Exposure (EAE) to standard Backpropagation (BP) as established in Proposition 2.1, and (2) the favorable memory scaling bounded in Lemma 2.2.

### 3.1 Experimental Setup

**Model Architecture.** We instantiate an autoregressive transformer language model [7] containing approximately 60M parameters. The architecture is configured with $L = 12$ layers, a hidden dimension of $D = 512$, 8 query heads, and 4 key/value heads (employing Grouped-Query Attention). The feed-forward layers utilize a SwiGLU activation with an expansion dimension of $D_{inner} = 1408$. We apply Rotary Position Embeddings (RoPE), RMSNorm, and tie the input embedding and final projection weights ($\mathbf{W}_{emb} \equiv \mathbf{W}_{head}$).

**Dataset and Optimization.** All models are trained on the English split of the `openbmb/Ultra-FineWeb` dataset [8]. Optimization is performed using [9] with a peak learning rate of $4 \times 10^{-4}$, weight decay of 0.1, and $\beta = (0.9, 0.95)$. We employ a cosine learning rate decay schedule down to $10\%$ of the peak, featuring a 200-step linear warmup. The models are trained over 800 optimization steps. To test the boundary scaling dynamics, we use a micro-batch size of 8 and 8 gradient accumulation, yielding an effective batch size of 64 sequences (with a maximum sequence length of 1024), processing approximately $65,536$ tokens per step.

**Hardware and Baselines.** Experiments are executed on a single NVIDIA T4 GPU utilizing Automatic Mixed Precision (AMP). We compare standard PyTorch Backpropagation (Baseline) against our foundational Explicit Adjoint Exposure (EAE, Algorithm 1) and the Memory-Optimized variant (EAE-GC, Algorithm 2).

### 3.2 Verification of Exact Adjoint Equivalence

To empirically validate Proposition 2.1, we compare the end-to-end learning dynamics and final convergence metrics of the standard backpropagation baseline against both variants of EAE. If the discrete Hamiltonian formulation and boundary conditions are precise, the optimization trajectory must be identical.

As summarized in Table 1, the EAE framework strictly mirrors the convergence of the standard baseline. The Baseline model converges to an evaluation loss of 5.0403 (Perplexity: 154.51). In perfectly mirrored trajectories, both EAE and EAE-GC converge to an evaluation loss of 5.0396 (Perplexity: 154.41).

Table 1: Performance and memory profiling for standard Backpropagation (Baseline) versus the proposed Explicit Adjoint Exposure methods. Results are reported after 800 optimization steps.

| METHOD | EVAL LOSS ↓ | PERPLEXITY ↓ | VRAM (GB) ↓ |
|---|---|---|---|
| BASELINE (STANDARD BP) | 5.0403 | 154.51 | 8.3 |
| EAE (ALG. 1) | 5.0396 | 154.41 | 9.0 |
| EAE-GC (ALG. 2) | **5.0396** | **154.41** | **7.7** |

The marginal discrepancy between the baseline and EAE ($\Delta_{\mathcal{L}} \approx 0.0007$) is fundamentally an artifact of floating-point non-associativity. Standard BP accumulates gradients recursively through the monolithic compiled graph, whereas EAE materializes the dual momenta explicitly in user manipulable way, subtly altering the specific order of FP16/BF16 matrix addition operations across gradient accumulation steps. Crucially, EAE and EAE-GC yield exactly identical losses (5.0396), confirming that the aggressive local garbage collection in Algorithm 2 alters strictly the lifetime of the computational graph, not its mathematical content.

### 3.3 Memory Optimization and Tape Decoupling

In Lemma 2.2, we posited that EAE-GC achieves an improved spatial complexity. The standard baseline requires a memory of 8.3 GB VRAM. Conversely, our Memory-Optimized Explicit Adjoint Exposure (EAE-GC) requires a VRAM of only 7.7 GB. By executing local Hamiltonian graphs and aggressively using garbage collection at each block boundary, EAE-GC drops 0.6 GB of redundant tape allocations. While a 600 MB reduction is notable on a compact 60M parameter architecture. As transformer architectures scale toward hundreds of layers and much wider intermediate expansions, the improvement in memory footprint provides immense scalability benefits with the added flexibility of accessible adjoint states.

## 4 Conclusion

In this work, we introduced Explicit Adjoint Exposure (EAE). By restructuring the standard, monolithic global AD tape into a sequence of isolated, block-local AD sweeps over localized Discrete Layer Hamiltonians, we successfully

decoupled the global computational graph. We theoretically established and empirically verified that EAE preserves exact analytical gradient equivalence to standard backpropagation, enforcing boundary conditions that ensure equivalent optimization trajectories and final convergence within standard floating point precision limits.

Crucially, this decoupling resolves fundamental memory and observability bottlenecks inherent to continuous reverse-mode graphs. We demonstrated that our memory-optimized variant, EAE-GC, algorithmically bounds the spatial complexity of the active auto-differentiation tape to $\mathcal{O}(ND_{inner})$. By instantaneously deallocating local Hamiltonian graphs, EAE-GC achieves an $\mathcal{O}(1)$ active tape scaling with respect to continuous network depth $L$, successfully reducing the total backward memory footprint to an additive $\mathcal{O}(NDL) + \mathcal{O}(ND_{inner})$ bound. Our empirical evaluations on an autoregressive transformer architecture confirm this theoretical limit, matching the baseline training dynamics while safely eliminating redundant VRAM allocations.

Beyond spatial efficiency, the fundamental contribution of EAE is the explicit materialization of the costates at block boundaries. Moving these dual momenta from an opaque compiler backend into user-accessible memory provides a mathematically sound primitive for advanced optimization paradigms. Future research will leverage this decoupled architecture to investigate localized adjoint quantization and sparsification to alleviate communication bottlenecks in distributed training, dynamic intermediate regularization, and the deployment of auxiliary synthetic costate predictors. EAE establishes a scalable, transparent, and highly manipulable foundation for next-generation deep learning optimization.

## References


[1] Paul Werbos. Beyond regression: New tools for prediction and analysis in the behavioral sciences. *PhD thesis, Committee on Applied Mathematics, Harvard University, Cambridge, MA*, 1974.

[2] David B Parker. Learning-logic: Casting the cortex of the human brain in silicon, 1985.

[3] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[4] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. Backpropagation applied to handwritten zip code recognition. *Neural computation*, 1(4):541–551, 1989.

[5] Lev Semenovich Pontryagin. *Mathematical theory of optimal processes*. Routledge, 2018.

[6] Arthur Earl Bryson. *Applied optimal control: optimization, estimation and control*. Routledge, 2018.

[7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[8] Yudong Wang, Zixuan Fu, Jie Cai, Peijun Tang, Hongya Lyu, Yewei Fang, Zhi Zheng, Jie Zhou, Guoyang Zeng, Chaojun Xiao, Xu Han, and Zhiyuan Liu. Ultra-FineWeb: Efficient data filtering and verification for high-quality llm training data, 2025.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization, 2019.